

Response to Reviewer Comments on Parallelization Implementation

1 Response to Comment 2: “Optimized Parallel Implementation” and “Highly Parallelized Module”

We appreciate the reviewer’s request for clarification regarding our parallelization terminology and implementation. We provide below a detailed technical explanation of our parallel implementation strategy.

1.1 Nature of the Parallelization

The reviewer’s assessment that “parallelization is only over (trivially parallel) independent displacements for numerical gradient calculations” is partially correct but underestimates the computational sophistication of our implementation. While we acknowledge that displacement-level parallelism is indeed *embarrassingly parallel* in nature, we employ the term “highly parallelized” to emphasize three key aspects that extend beyond simple task-level parallelism.

First, our module provides **comprehensive integration** with multiple quantum chemistry codes through a unified parallel framework. Unlike standalone numerical gradient implementations that are typically code-specific, our implementation seamlessly integrates with Gaussian, MRCC, Molpro, and ORCA through a standardized External Interface protocol. This unified approach allows researchers to leverage parallel gradient calculations across different electronic structure methods without reimplementing parallelization logic for each code.

Second, we incorporate **advanced symmetry exploitation** through a sophisticated symmetry engine (`NonAbelianSymmetryEngine` in `elecext/symmetry_engine.py`) that automatically detects and exploits molecular point group symmetry to minimize the number of required displacements. For molecules with non-Abelian point groups such as C_{3v} , D_{3h} , and D_{6h} , this reduces computational requirements by 50–75% before any displacement-level parallelism is applied. This symmetry-aware gradient assembly involves automatic extraction of symmetry operations from Gaussian frequency calculations, inference of nuclear permutations and 3×3 transformation matrices from force patterns when explicit operations are unavailable, vector-based gradient propagation through symmetry operations, and multi-pass gradient assembly for constrained components. This preprocessing step represents a significant computational optimization that is orthogonal to the parallelization strategy and justifies the characterization of our approach as “highly” optimized.

Third, the implementation provides **granular resource management** with fine-grained control over computational resource allocation between the parallel orchestration layer and individual electronic structure calculations. This aspect is detailed in the Resource Management section below and represents a key architectural advantage over monolithic gradient implementations.

1.2 Technical Implementation Details

Our implementation uses Python’s `concurrent.futures.ThreadPoolExecutor` to manage parallel execution of displacement calculations, as shown in the following code excerpt from `elecext/parall.py` (lines 675–751):

```
1 def run_energy_tasks_in_parallel(geometries_to_calculate ,
2                                 displacement_info ,
3                                 hooks , max_workers=None):
4     with ThreadPoolExecutor(max_workers=max_workers) as executor:
```

```

5         fut_map = {
6             executor.submit(run_single_point_energy, task_id,
7                             geom, hooks,
8                             displacement_info.get(task_id)): task_id
9         for task_id, geom in geometries_to_calculate.items()
10    }
11    for fut in as_completed(fut_map):
12        # Process completed tasks...

```

Each worker thread executes a complete single-point energy calculation by first writing input files for the electronic structure code, then spawning the external program via `subprocess.run()` (line 332 of `parall.py`), subsequently parsing the output energy, and finally returning results to the main orchestrator.

Our architecture employs a **hybrid thread-process model**: while the orchestration layer uses threads through `ThreadPoolExecutor`, each thread spawns an independent *process* for the quantum chemistry calculation via the `subprocess` module. This hybrid approach provides several advantages. First, lightweight task scheduling is achieved via Python threads, which have minimal overhead compared to process creation. Second, complete process isolation is maintained for quantum chemistry codes, preventing any potential interference between concurrent calculations. Third, Python’s Global Interpreter Lock (GIL) limitations for CPU-intensive operations are circumvented, as the actual computational work occurs in separate subprocesses that do not share the GIL.

1.3 Resource Management and Control

A distinguishing advantage of our External Interface implementation is the **granular control over resource partitioning** between the parallel displacement orchestrator and individual electronic structure calculations. The `CentralExt` dispatcher automatically adjusts computational resources through dynamic scaling, as illustrated in the following code from lines 306–344:

```

1  # Memory: multiply by (1 + nthreads) to account for
2  # parallel overhead
3  # Threads: add nthreads to base threads
4  adjusted_mem = f"{base_mem_value}*_{(1+_nthreads)}_{mem_unit}"
5  adjusted_omp = str(int(mrcc_omp_procs) + nthreads)

```

Specifically, memory allocation is scaled by a factor of $(1 + N_{\text{parallel_workers}})$ to prevent memory contention between concurrent calculations. Thread and core distribution is managed by allocating $N_{\text{base_threads}} + N_{\text{parallel_workers}}$ to the electronic structure code while reserving workers for parallel orchestration. Additionally, program-specific parameters are adapted to different quantum chemistry codes; for instance, MRCC uses `omp_procs` and `mpi_procs` parameters, while other codes use `nprocs`.

This granular resource control allows users to optimize the trade-off between two competing parallelization strategies: intra-displacement parallelism (multi-threaded or MPI execution within each electronic structure calculation) versus inter-displacement parallelism (the number of simultaneous displacement calculations). For example, with 12 CPU cores available, users can choose among several strategies: Strategy A allocates 4 parallel workers with 3 cores each dedicated to the electronic structure code; Strategy B uses 2 parallel workers with 6 cores each; Strategy C employs 1 worker with all 12 cores for analytical gradient calculations combined with parallel numerical displacements for other components.

This flexibility is **not trivially achieved** in monolithic gradient implementations and represents a significant advantage of the External Interface architecture. Traditional implementations typically

fix the parallelization scheme at compile time or through rigid configuration files, whereas our approach allows dynamic resource allocation that can be optimized for each molecular system and computational environment.

1.4 System Architecture: Shared Memory vs. Distributed Memory

The core parallelization framework is implemented for **shared-memory systems**, using Python’s `ThreadPoolExecutor` and direct `subprocess` calls. This design assumes that all workers access a common filesystem, memory is shared (though not explicitly used for inter-worker communication), and no inter-node communication layer is required. This architecture is appropriate for the vast majority of quantum chemistry calculations for method development and small-to-medium molecule studies, which are performed on shared-memory workstations or single-node servers.

For users requiring distributed memory execution on HPC clusters with job schedulers such as SLURM or PBS, we provide interface scripts in the GitHub repository that adapt the framework to submit displacement calculations as independent batch jobs. These scheduler interface scripts replace direct `subprocess` calls with job submission commands (`sbatch` for SLURM, `qsub` for PBS) and implement asynchronous job status monitoring to coordinate distributed execution across multiple compute nodes. The interface handles job queue management, dependency tracking, and result collection, transforming the shared-memory orchestrator into a distributed workflow manager compatible with standard HPC environments.

Extension to container orchestration systems (Kubernetes, Docker Swarm) or cloud computing platforms (AWS, Google Cloud, Azure) would require additional modifications beyond the provided scheduler interfaces, including implementation of cloud-specific APIs for resource provisioning, cost optimization algorithms for billed computational resources, and network communication protocols for distributed task distribution. However, the scheduler interface scripts demonstrate that the framework’s modular architecture supports adaptation to distributed environments through well-defined extension points, maintaining the core benefits of code-agnostic gradient calculations while enabling scalability to cluster-scale computations when required.

2 Response to Comment 3: Nomenclature for Parallelization (Threads vs. Processes)

We thank the reviewer for identifying the inconsistency in our nomenclature. We acknowledge that our use of “NThreads” in Table S43 is **technically imprecise** and requires clarification.

2.1 Correct Technical Description

At the orchestration level, the parallel worker management uses Python threads through the `concurrent.futures.ThreadPoolExecutor` class. However, each thread spawns an independent *process* for the quantum chemistry calculation via `subprocess.run()` (or `subprocess.Popen` in some variants). Therefore, the parameter labeled as “NThreads” in Table S43 should be more precisely termed “**parallel workers**” or “**concurrent displacement calculations**”. Each “worker” is implemented as a Python thread that spawns subprocess-based calculations, resulting in a hybrid threading-multiprocessing model.

2.2 Timing Measurements: Wall-Clock Time vs. CPU Time

All reported timings in our manuscript are **wall-clock times** measured using Python’s `time.perf_counter()` function (as implemented in `elecext/profiling.py`, lines 20, 51, and 66):

```
1 start_time = time.perf_counter()
2 # ... perform calculation ...
3 duration = time.perf_counter() - start_time
```

This high-resolution timer measures elapsed real time (wall-clock), not accumulated CPU time. For parallel calculations, this distinction is critical: wall-clock time reflects the actual time experienced by the user from submission to completion, whereas CPU time would sum the computational effort across all cores. For numerical gradient calculations with N parallel workers, the relationship is approximately given by

$$t_{\text{wall}} \approx \frac{t_{\text{sequential}}}{N} \quad (\text{ideal scaling}) \quad (1)$$

while CPU time remains approximately constant:

$$t_{\text{CPU}} \approx t_{\text{sequential}} \quad (\text{independent of } N) \quad (2)$$

Our performance benchmarks report wall-clock speedups, which are the relevant metric for assessing practical computational efficiency and user productivity.

2.3 Parallelism Strategy and Efficiency

The reviewer correctly notes that multiple parallelization strategies can be combined, and we acknowledge that our implementation supports flexible combinations of these approaches. Strategy A employs intra-calculation parallelism, where single-point energy or analytical gradient calculations use multi-threaded programs (via OpenMP) or MPI-parallel codes. For example, MRCC can be executed with 3 OpenMP threads or 3 MPI processes to parallelize the internal correlation calculation.

Strategy B, which represents our primary approach, uses inter-displacement parallelism where individual displacements are computed by independent workers, each running single-threaded or minimally-threaded quantum chemistry calculations. This approach avoids parallel inefficiencies that can arise within quantum chemistry codes when scaling to larger core counts, potentially achieving lower total CPU time due to better CPU utilization and reduced parallel overhead.

Strategy C combines both approaches through our dynamic resource allocation mechanism. For instance, with 12 available cores, one could configure 4 parallel workers with each MRCC calculation using 3 threads, achieving parallelism at both the displacement level and within each calculation. This hybrid strategy is configurable via resource adjustment parameters in **CentralExt** (lines 306–344).

The optimal strategy depends on several factors including molecular size (which affects memory scaling and cache utilization), the electronic structure method (different methods have varying parallel scaling efficiency), hardware architecture (NUMA effects, cache hierarchy, memory bandwidth), and system load and contention from other processes. Our implementation provides the **flexibility to explore these trade-offs** through user-configurable resource parameters, rather than imposing a fixed parallelization scheme that may be suboptimal for certain systems or computational environments.

2.4 Revised Nomenclature for Manuscript

We propose the following corrections to address the nomenclature inconsistencies identified by the reviewer. For Table S43, the caption should be revised from “Performance scaling with NThreads parallel processes” to “Performance scaling with N parallel workers (Python threads spawning subprocess-based calculations)” to accurately reflect the hybrid threading-multiprocessing model.

In the main text (page 3, line 37), we will change “a newly developed, highly parallelized module” to “a newly developed parallel gradient module with symmetry exploitation and configurable resource management” to more precisely describe the implementation without overstating the sophistication of the displacement-level parallelism.

In the abstract, “optimized parallel implementation” will be revised to “parallel implementation with automatic symmetry reduction and granular resource control” to emphasize the distinctive features of our approach (symmetry preprocessing and flexible resource management) rather than using generic parallelization terminology.

Code References:

- Parallel execution: `/elecext/parall.py:675--751` (`run_energy_tasks_in_parallel`)
- Subprocess calls: `/elecext/parall.py:332` (`subprocess.run`)
- Resource adjustment: `/Executables/CentralExt:306--344`
- Timing measurements: `/elecext/profiling.py:20,51,66` (`time.perf_counter`)
- Symmetry engine: `/elecext/symmetry_engine.py:254--956` (`NonAbelianSymmetryEngine`)